CA326 Year 3 Project - FootStat Technical Specification

Student Names: Alen Joy - Student ID: 18313576
Jaime de Vivero Woods - Student ID: 19447494

Project Title: FootStat - Collecting and analysing European football league statistics

Supervisor: Marija Bezbradica

Document type: Technical Specification

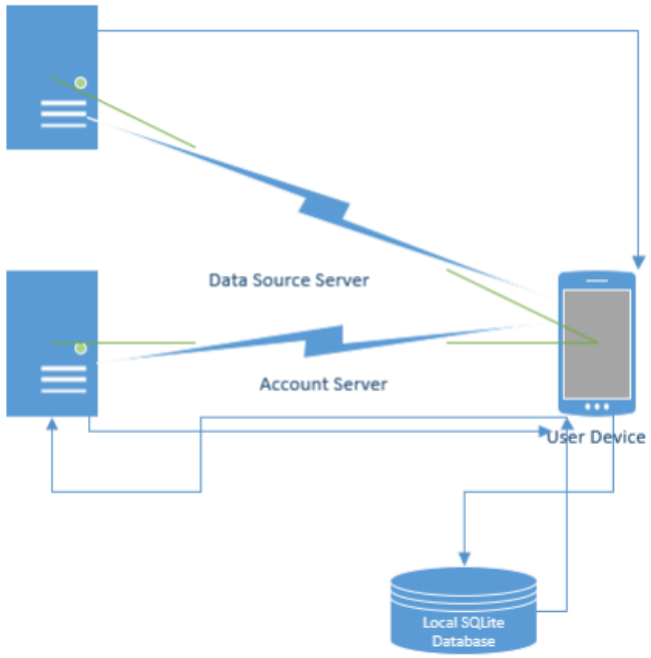
Date of completion: 04/03/2022

Overview

Our project idea was to develop an Android app which took publicly available statistics on major European soccer leagues from sports websites and displayed them in various formats and layouts. This was one of the staff project ideas from Mark Roantree. We wanted the functions available for the app to view the results of the leagues, the performance of the teams in past games and view the rankings of teams and players according to their results from applying various algorithms to help determine their relative strength.

Our final app does most of the functions we set out to do. Our app can display the results from the leagues, the performance of the teams in these leagues, the elo's of all of the teams have been calculated using a series of mathematical functions using the data from the leagues. This elo determines the strength of the teams and can be used for comparison.

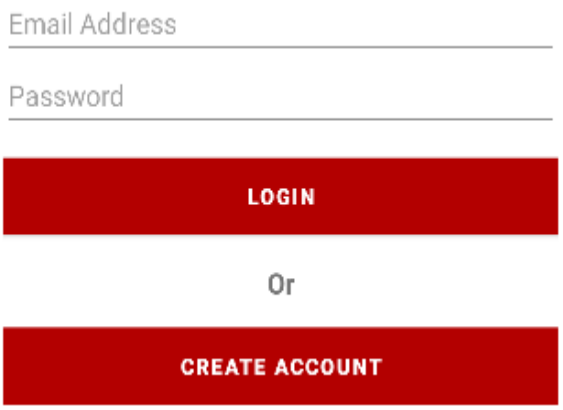
High-level design

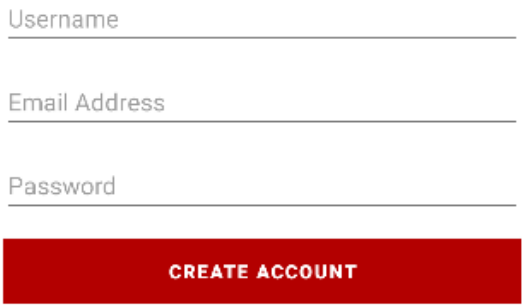


(Fig 1)

Figure 1 represents the basic design of our application at a high level. On launch, it displays a login screen where a user can either enter credentials if they already have an account or create a new account. These credentials are checked on the Account Server, a Firebase collection containing all usernames, their generated IDs and their email addresses. If the login is successful, the user will be sent to the home page of the app.

(Fig 2) **(Fig 3)**

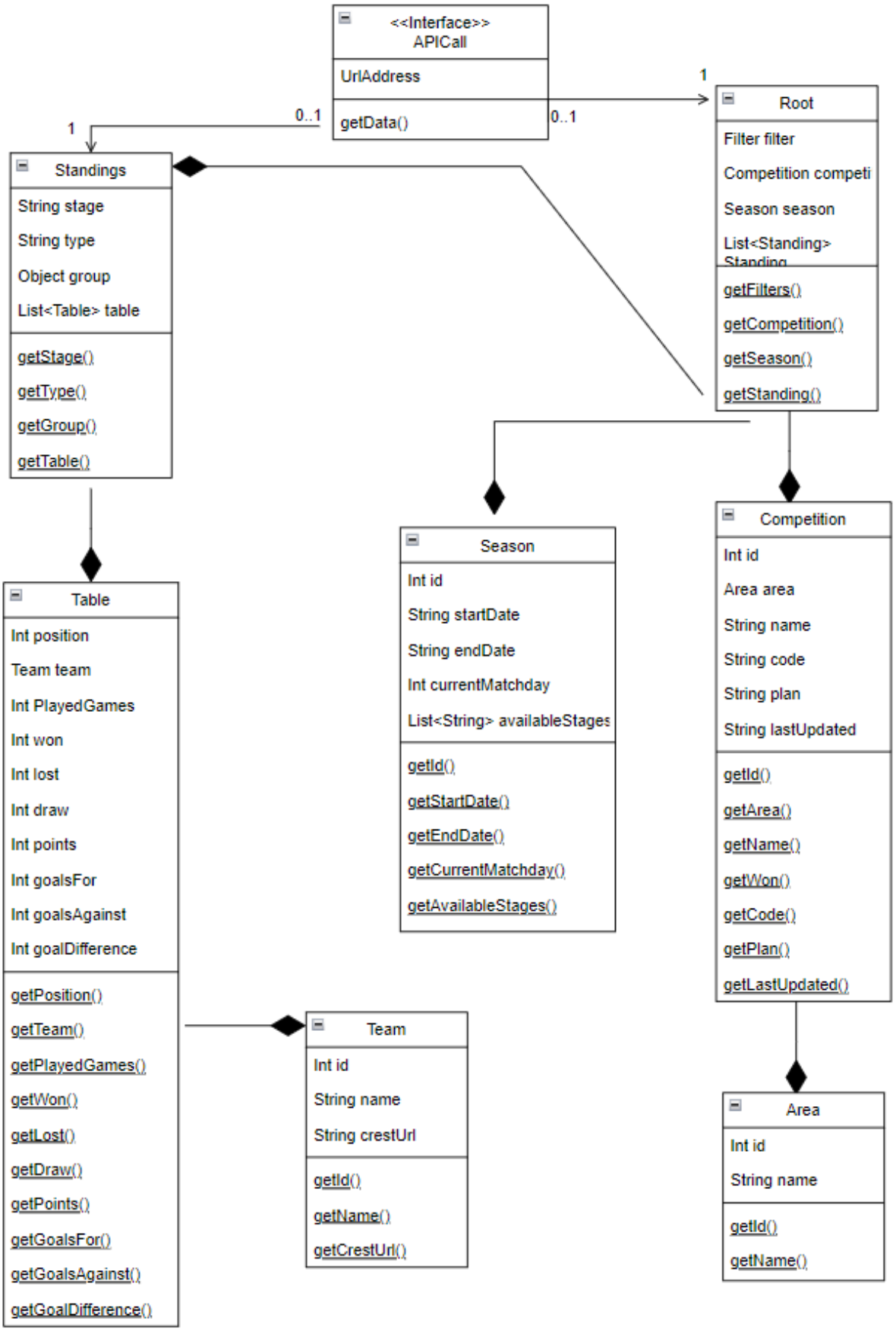




From the homepage, a user can access a number of activities that involve retrieving and displaying football league data, like scores, teams, matches etc. When the user accesses one of these activities, the app communicates with the Data Source Server, which holds the JSON data from an API that collects football statistics. The data for the earlier mentioned activities is retrieved from the JSON, either using

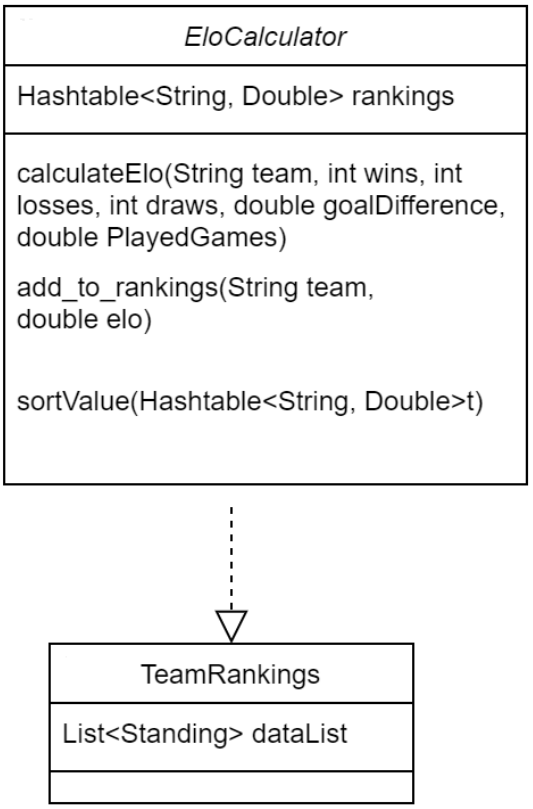
objects built from them or databases. The databases illustrated in fig 1 are created using object models of the JSON data.

Some of the class diagrams and object models used in the application are shown below.

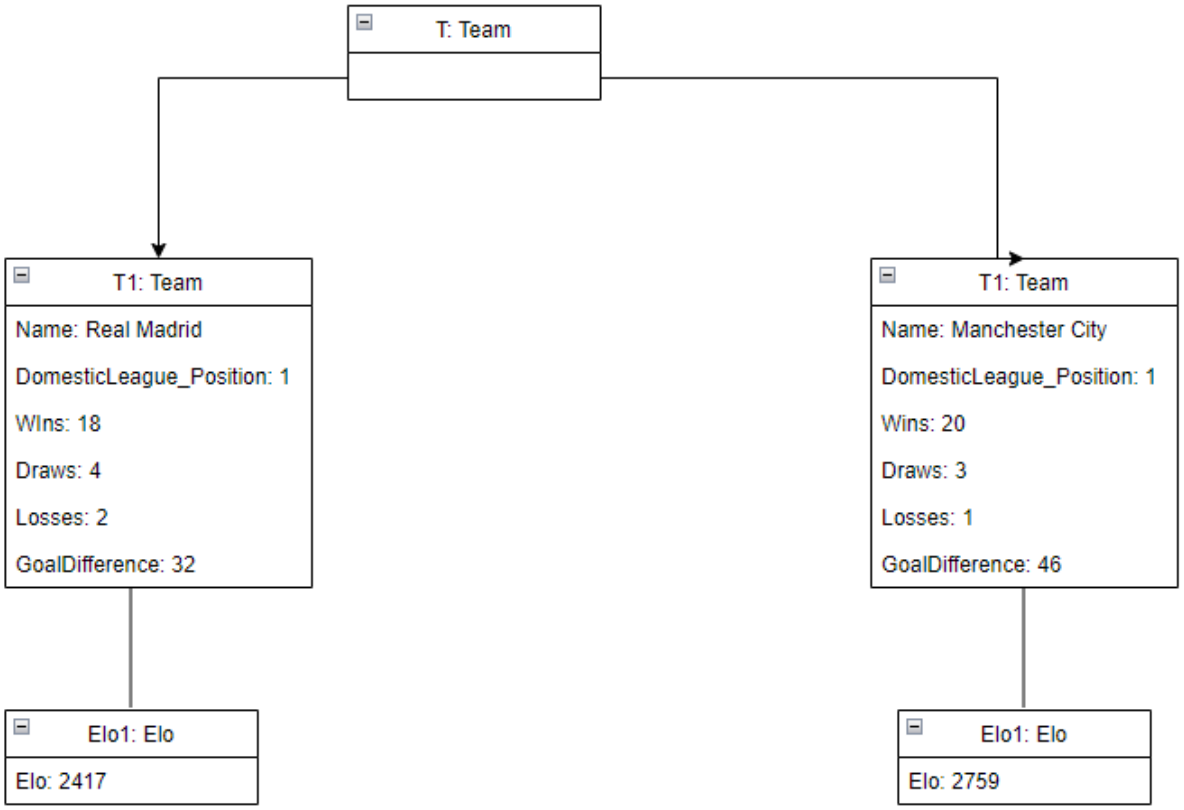


(Fig 4)

Class diagram for Elo calculator:

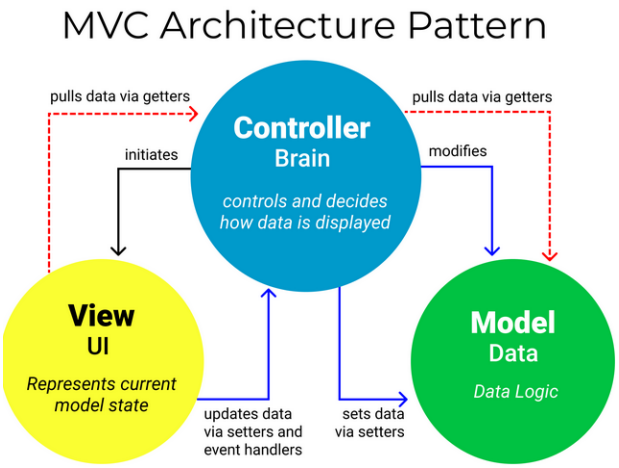


Object Models:



(Fig 5)

System Architecture



(Fig 6)

For our project, we have tried to follow the MVC (- Model View Controller) architecture as much as possible. This is in an effort to separate the UI, data processing and storage as much as possible.

Implementation

JSON Handling and Object Modelling

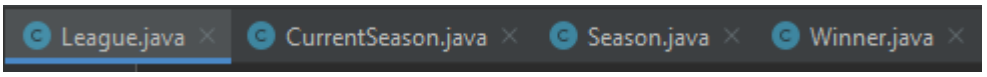
The first task we decided to undertake was to retrieve JSON data from our API, and deserialize it to create objects for use in the database. These objects would also be used for the ELO rating algorithm we planned to develop to rank the teams. After we had performed testing to prove the integrity of the objects, we would move on to developing a database to store this data. We would hit endpoints from the API found here: https://www.football-data.org/ to retrieve the data.

Interpreting the data proved to be a significantly more complex task than we expected after we realised that the JSON data could contain hundreds of entries that were nested to various levels. The data also contained an abundance of null values that had to be filtered out before being added to a database. For this reason, we were required to design multiple object models (known as POJOs) for each nested level to unpack the data, and these were tested thoroughly so that each method worked as expected.

```
{
    "id": 2002,
    "area": {
        "id": 2088,
        "name": "Germany"
    },
    "name": "Bundesliga",
    "code": "BL1",
    "emblemUrl": null,
    "plan": "TIER_ONE",
    "currentSeason": {
        "id": 742,
        "startDate": "2021-08-13",
        "endDate": "2022-05-14",
        "currentMatchday": 25,
        "winner": null
    },
    "seasons": [
        {
            "id": 742,
            "startDate": "2021-08-13",
            "endDate": "2022-05-14",
            "currentMatchday": 25,
            "winner": null
        },
        {
            "id": 599,
            "startDate": "2020-09-18",
            "endDate": "2021-05-22",
            "currentMatchday": 34,
            "winner": {
                "id": 5,
                "name": "FC Bayern München",
                "shortName": "Bayern",
                "tla": "FCB",
                "crestUrl": "https://crests.football-data.org/5.svg"
```


(Fig 7)

Above is an example of the JSON data we retrieved from one of the endpoints. Most other endpoints had similar levels of nesting and were equally complex to parse. To implement the model for the JSON above, we were required to create the classes shown below.



For fields that were not relevant or important, like “area”, we didn’t make models for them.

```
AreaTest
CompetitionTest
EloCalculatorTest
RootTest
SeasonTest
StandingTest
TableTest
TeamTest
```

(Fig 8)

These are the unit tests we made with JUnit5 to test the JSON classes made to unpack the data.

(Fig 9)

```
ublic class StandingTest {
    List<Table> table1 =  new ArrayList<~>();//Table(2,team1, 24, 16, 6, 2, 54, 48, 16, 32);
    Standing standing1 = new Standing( stage: "REGULAR_SEASON", type: "TOTAL", group: "null", table1);

    @Test
    public void givenData_getStageTest() {
        String actual_Stage = standing1.getStage();
        String expected_Stage = "REGULAR_SEASON";
        Assert.assertEquals(actual_Stage, expected_Stage);
    }

    @Test
    public void givenData_getTypeTest() {
        String actual_Type = standing1.getType();
        String expected_Type = "TOTAL";
        Assert.assertEquals(actual_Type, expected_Type);
    }

    @Test
```

This is one of the unit tests we used for the table heading in the JSON data.

After we made object models for the JSON, the next step was deserializing the raw JSON data to create objects from them. We achieved this by using Gson, which inserted the values for the object attributes.

```
League premier = new Gson().fromJson(file2021, League.class);
formattedLeague(premier);
allLeagues.getLeagueList().add(premier);
```

(Fig 10)

Database Creation

The SQLite databases were created using Room. Room allows for direct creation of tables by using objects as entities.

```java
@Entity(tableName = "league_table")
public class League {

    @PrimaryKey(autoGenerate = true)
    @ColumnInfo(name = "id")
    private Integer id;

    @ColumnInfo(name ="league_name")
    private String name;

    @ColumnInfo(name ="league_code")
    private String code;

    @ColumnInfo(name = "emblem_url")
    private String emblemUrl;

    @Ignore
    private CurrentSeason currentSeason;
```

(Fig 11)

Above is an example of its usage. Each attribute of League will have a row in the table created for it. Rows we don't want to include can be ignored with the @ignore annotation.

```java
    // CRUD ops
    @Insert(onConflict = OnConflictStrategy.IGNORE)
    void insert(League league);

    @Query("DELETE FROM league_table")
    void deleteAll();

    @Query("SELECT * FROM league_table ORDER BY league_name ASC")
    LiveData<List<League>> getAllLeagues();
}
```

(Fig 12)

After creating the entities, we need to use a data access object (DAO) to perform CRUD operations. To make the database operations as efficient as possible and to keep the main thread free so it doesn't slow down interaction with the UI, we employed multithreading by creating a threadpool of database executors.

```
// Creating a thread pool of executors to perform database operations
// Doing this keeps the main thread free to handle user interaction and other main activities
// This will prevent users experiencing perceivable slowdown on most devices

public abstract LeagueDao leagueDao();
public abstract ConvertedWinnerDao winnerDao();
public static final int THREADS = 4;

public static volatile LeagueRoomDatabase INSTANCE;
public static final ExecutorService databaseWriteExecutor =
        Executors.newFixedThreadPool(THREADS);
```

(Fig 13)

The database was tested using Android Studio's built in database inspector.

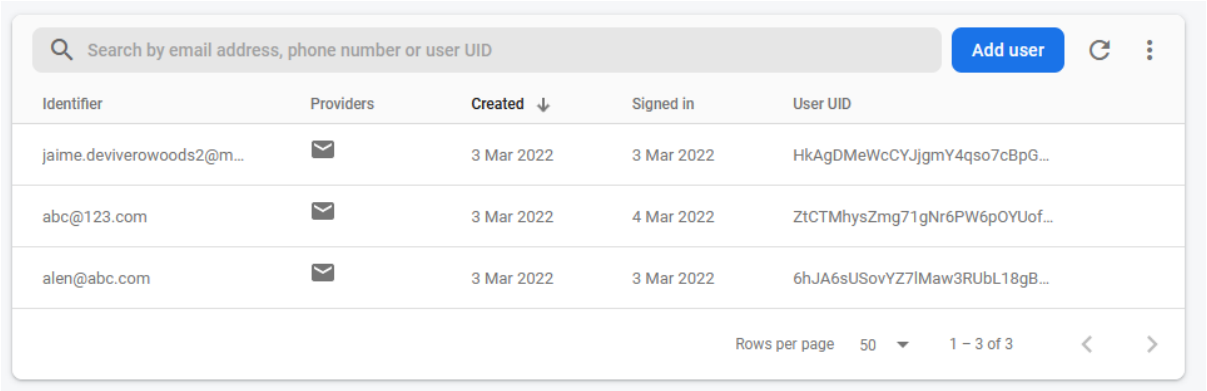
ELO Algorithms

We then implemented the elo algorithms. To do this we needed wins, losses, draws, goalDifference and played games. To calculate the elo, the formula we used was *elo = (((wins*100)+(draws*50)) + (Goal difference / Games Played)*10)*. We added the team name and elo for each team to a HashTable, so it was easier to reference and print. We then made two methods, one to add the elo to the HashTable and another to sort the HashTable. All of these methods were defined in the class "EloCalculator.java".

```java
public class EloCalculator {
    private static double elo;
    private static Integer rounded_elo;

    //double elo;

    //private ArrayList<String> rankings = new ArrayList<String>();
    static Hashtable<String, Double> rankings = new Hashtable<>();

    public static int calculateElo(String team, int wins, int losses, int draws, double goalDifference, double playedGames){
        elo = (wins * 100) + (draws * 50) + (losses * 0);
       // double loss_ratio = (losses / playedGames) * 100;
        elo = elo + (goalDifference / playedGames) * 10;
        rounded_elo = (int) elo;
        return rounded_elo;
    }

    public static Hashtable<String, Double> add_to_rankings(String team, double elo){
        String otpt = (team + " : " + elo);
        rankings.put(team, elo);
        rankings = sortValue(rankings);
        return rankings;
    }

    public static Hashtable<String, Double> sortValue(Hashtable<String, Double> t){

        //Transfer as List and sort it
        ArrayList<Map.Entry<String, Double>> l = new ArrayList(t.entrySet());
        Collections.sort(l, new Comparator<Map.Entry<String, Double>>(){

            public int compare(Map.Entry<String, Double> o1, Map.Entry<String, Double> o2) {
                return o1.getValue().compareTo(o2.getValue());
            }});
        return t;
    }
```

Firebase authentication

Next we created a login screen by using Firebase authentication services. We used a custom Firebase database to store the user credentials and retrieve and check them. We used email authentication for our system. Here is an example of users stored in the database.



Testing

After performing unit testing using JUnit, we performed integration testing by performing certain tasks and using the logcat and debugger to make sure there were no issues.

We followed the Bottom-Up testing approach for integration testing and tested lower level modules such as the module which printed the standings from the parsed JSON first. This test was then used when using thosed stats parsed to perform the elo algorithms.

This approach helped find any faults along the way. One error that we found while performing tests at the lower level modules was that the module which calculated the elo took in an integer for the group position, but was given a string by the module which parsed the json data. We realised this and fixed it. This error would have been harder to find if we followed another testing approach such as the big bang testing approach as we would have had to go back to our unit tests and would have had to perform more tests.

After we developed the login, we went through system and user testing. We performed system testing by running the app and performing all the actions possible, and in combination with each other. For example, we checked the Premier league results, then the elo ratings for the teams and then the results for the Bundesliga.

Once we performed our system testing, we performed user testing by having friends and family members use our app. We chose to do black box testing for our user testing, so that the user would use the functions without seeing any code or learning about the functionalities of the modules that made the app.

Problems & Solutions

When developing the app, one of the largest problems we had was referencing data that was nested to various levels. We struggled to define the objects that were nested in other objects. For example, defining arrays nested in an array nested in JSON data was hard for us to define. The way we solved this problem was by making extensive tests for each object defined as we moved deeper into the nested data. As we did this and saw the results, we were able to specify the correct objects to interpret the data.

Another problem we encountered was printing the team data and the elo for each respective team. Originally, we added the elo values to an array, but when printing the elo of teams, it got difficult to manage what value in the array belonged to which team. We solved this problem by using a HashTable of String and double. This made printing and referencing much easier.

Creation of the databases was also a major challenge. There were many steps involved in the process and it was very error prone. One of the main difficulties was having to create new objects entirely just to add them to the database, even though they use the same attributes as are in the objects. This is because, to create a table of attributes belonging to 2 different objects, you normally need one object to inherit from the other. However, Gson doesn’t support deserializing objects with inheritances so this isn’t possible.

The next challenge was the authentication system. There was a huge amount of code required to be written for basic operations such as logging in and there were many situations that needed to be handled, like if a user only enters a username instead of both credentials.